

□ Threading & Coroutines en Python

Le support est une propriété de **Bechir Bejaoui**

1 □ Pourquoi la concurrence ?

Imaginez un cuisinier qui prépare un repas. En mode **séquentiel**, il met l'eau à bouillir, **attend** qu'elle bout, puis coupe les légumes, **attend** qu'ils cuisent, puis fait la sauce. Chaque étape attend la précédente —il perd un temps fou à ne rien faire.

En mode **concurrent**, il lance l'eau, et pendant qu'elle chauffe il coupe les légumes. Il **fait plusieurs choses en même temps** —ou du moins, il ne reste jamais inactif.

C'est exactement le problème qu'on résout en programmation.

```
import time
```

```
# Simulation : chaque tâche "attend" une réponse (base de données, réseau...)
def telecharger(fichier):
    print(f" Début téléchargement : {fichier}")
    time.sleep(2) # on attend la réponse — Le CPU ne fait RIEN pendant ce temps
    print(f" □ Terminé : {fichier}")
```

```
debut = time.perf_counter()
```

```
# Mode séquentiel : on attend la fin de chaque téléchargement
telecharger("rapport.pdf")
telecharger("données.csv")
telecharger("images.zip")
```

```
print(f"\n□ Temps total : {time.perf_counter() - debut:.1f}s")
# □ Temps total : 6.0s → on a attendu 3 × 2s sans rien faire
```

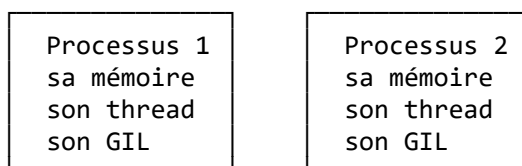
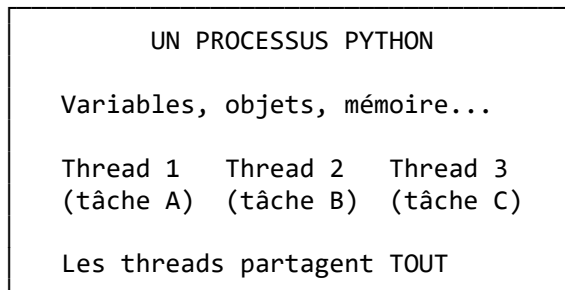
- **time.sleep(2)** simule une opération lente : lecture en base, appel réseau, écriture fichier —le CPU attend sans travailler
 - **6 secondes pour 3 fichiers** : chaque fichier attend que le précédent soit fini —inacceptable en production
 - **la concurrence** va permettre de lancer les 3 téléchargements en même temps → temps total $\approx$ 2s au lieu de 6s
-

Les 3 outils de concurrence en Python :

Outil	Quand l'utiliser	Exemple concret
Threading	Opérations I/O avec code classique	Téléchargements, lectures DB
asyncio	I/O massif, nombreuses connexions	Serveur web, 1000 appels API
multiprocessing	Calcul intensif CPU	Machine learning, encodage vidéo

2 □ Threads vs Processus

Le **thread** et le **processus** fournissent deux façons de faire tourner du code en parallèle. La différence clé : ce qu'ils **partagent**.



Processus isolés — ne partagent RIEN

- **Thread** → Un **thread** est simplement **un chemin d'exécution supplémentaire dans le même processus**.
- **Processus** → Un **processus** est une **instance d'un programme avec sa propre mémoire**

3 □ Exécution séquentielle (point de départ)

Avant d'introduire les threads, mesurons précisément ce que coûte l'approche séquentielle. C'est notre **référence**.

```
import time

def preparer_commande(numero):
    print(f" Commande #{numero} : en cours...")
    time.sleep(1)  # simule une opération lente (appel DB, réseau...)
    print(f" Commande #{numero} : ☑ prête")

debut = time.perf_counter()

# 3 commandes traitées l'une après l'autre
preparer_commande(1)
preparer_commande(2)
preparer_commande(3)

print(f"\n☑ Temps total : {time.perf_counter() - debut:.1f}s")

Commande #1 : en cours...
Commande #1 : ☑ prête
Commande #2 : en cours...
Commande #2 : ☑ prête
Commande #3 : en cours...
Commande #3 : ☑ prête
```

☑ Temps total : 3.0s

- **3 secondes** pour 3 commandes d'1 seconde chacune —le temps s'additionne
- **#2 attend que #1 soit finie** même si les deux opérations sont indépendantes —pure perte
- **objectif** : avec les threads, viser ~1 seconde pour les 3 commandes

4 □ Introduction au Threading

Un **thread** permet de lancer une fonction en arrière-plan, sans bloquer le reste du programme. Python fournit `threading.Thread` pour ça.

```
import time
import threading

def preparer_commande(numero):
```

```

    print(f"  Commande #{numero} : en cours...")
    time.sleep(1)
    print(f"  Commande #{numero} : ☑ prête")

debut = time.perf_counter()

# Créer 3 threads — un par commande
t1 = threading.Thread(target=preparer_commande, args=(1,))
t2 = threading.Thread(target=preparer_commande, args=(2,))
t3 = threading.Thread(target=preparer_commande, args=(3,))

# .start() Lance le thread — retourne immédiatement sans attendre
t1.start()
t2.start()
t3.start()

# .join() attend que CE thread soit terminé avant de continuer
t1.join()
t2.join()
t3.join()

print(f"\n☑ Temps total : {time.perf_counter() - debut:.1f}s")

Commande #1 : en cours...
Commande #2 : en cours...
Commande #3 : en cours...
Commande #1 : ☑ prête
Commande #2 : ☑ prête
Commande #3 : ☑ prête

☑ Temps total : 1.0s ← 3× plus rapide !

```

- **Thread(target=f, args=(...))** : crée un thread qui va appeler f avec les arguments —le thread n'est pas encore démarré
- **.start()** : lance le thread en arrière-plan et **retourne immédiatement** —les 3 commandes démarrent presque en même temps
- **.join()** : dit au programme principal "*attends que ce thread soit fini avant de continuer*"—sans ça, le programme pourrait se terminer avant les threads
- **l'ordre d'affichage peut varier** : les threads s'exécutent en parallèle, l'OS décide qui parle en premier

Version avec une classe (style orienté objet) :

```

import time
import threading

```

```

class CommandeThread(threading.Thread):

    def __init__(self, numero):
        super().__init__()
        self.numero = numero

    def run(self):
        # run() est appelée automatiquement par .start()
        print(f"  Commande #{self.numero} : en cours...")
        time.sleep(1)
        print(f"  Commande #{self.numero} : ☑ prête")

```

```

threads = [CommandeThread(i) for i in range(1, 4)]
for t in threads: t.start()
for t in threads: t.join()

```

- **hériter de `threading.Thread`** : utile quand le thread a besoin de **garder un état** (attributs, résultats)
- **`run()`** : la méthode à redéfinir —c'est elle qui sera exécutée dans le thread
- les deux styles (avec `target=` ou avec héritage) produisent le même résultat —choisissez selon la complexité

5 ☐ Race Condition —le bug silencieux

Voici le piège le plus dangereux du threading. Quand deux threads lisent et modifient **la même variable en même temps**, le résultat devient imprévisible.

```

import threading

compteur = 0  # variable partagée entre les deux threads

def incrementer():
    global compteur
    for _ in range(100_000):
        compteur += 1  # ☒ pas atomique : lire → ajouter 1 → écrire

t1 = threading.Thread(target=incrementer)
t2 = threading.Thread(target=incrementer)

t1.start()
t2.start()
t1.join()
t2.join()

```

```
print(f"Résultat attendu : 200 000")
print(f"Résultat obtenu : {compteur}") # souvent < 200 000 !
```

Ce qui se passe réellement

Sans verrou, l'exécution peut devenir :

Thread A lit compteur
→ compteur = 10

Thread B lit compteur
→ compteur = 10

Thread A écrit 11

Thread B écrit 11

Résultat final :

11 au lieu de 12

Une **incrémentation est perdue**.

Python exécute plusieurs instructions internes mais **le GIL peut empêcher l'interruption au mauvais moment**, donc les threads passent souvent l'un après l'autre.

6 □ Lock —protéger les données partagées

Un **Lock** (verrou) garantit qu'une seule section de code s'exécute à la fois. Les autres threads **attendent leur tour**.

```
import threading

compteur = 0
lock = threading.Lock()

def incrementer():
    global compteur

    for _ in range(10000):
        with lock:
            compteur += 1
```

```
t1 = threading.Thread(target=incrémenter)
t2 = threading.Thread(target=incrémenter)

t1.start()
t2.start()

t1.join()
t2.join()
```

```
print("Résultat :", compteur)
```

Cette fois le résultat sera **toujours correct**.

Ce qui se passe avec le Lock :

- **with lock:** : acquiert le verrou à l'entrée, le libère automatiquement à la sortie —même en cas d'exception
- **section critique** : le code entre with lock: ne peut être exécuté que par **un thread à la fois**
- **coût** : le lock ralentit légèrement (les threads attendent) —ne l'utiliser que sur les parties réellement partagées

7 □ Semaphore —limiter le nombre de threads simultanés

Là où Lock n'autorise que **1 thread à la fois**, Semaphore(n) en autorise **n simultanément**. Utile pour ne pas surcharger une ressource (API, connexions DB).

```
import threading
import time

# On simule une API externe qui n'accepte que
# 2 connexions simultanées
semaphore = threading.Semaphore(2)

def appeler_api(worker_id):
    print(f" Worker {worker_id} : en attente d'un slot...")
    with semaphore:
        print(f" Worker {worker_id} : ☑ connecté")
        time.sleep(1)
        print(f" Worker {worker_id} : ☑ terminé")

# 5 workers veulent accéder à l'API en même temps
threads = [threading.Thread(target=appeler_api, args=(i,))
```

```

for t in threads: t.start()
for t in threads: t.join()

```

Worker 1 : en attente d'un slot...
 Worker 2 : en attente d'un slot...
 Worker 3 : en attente d'un slot...
 Worker 1 : ☑ connecté ← max 2 simultanément
 Worker 2 : ☑ connecté
 Worker 1 : ☑ terminé
 Worker 3 : ☑ connecté ← Worker 3 entre quand Worker 1 libère
 ...

- **Semaphore(2)** : compteur initialisé à 2 —chaque with semaphore décrémente, chaque sortie incrémente
- **quand le compteur atteint 0** : les threads suivants attendent qu'un slot se libère
- **cas d'usage typiques** : limiter les connexions à une DB, respecter le rate limit d'une API externe, pool de ressources

Quand utiliser Lock et quand utiliser Semaphore

Lock

À utiliser quand :

- **une seule exécution à la fois** doit entrer
- on protège une donnée critique
- on veut éviter toute modification concurrente

Semaphore

À utiliser quand :

- on autorise **plusieurs accès simultanés**
- la ressource supporte une capacité limitée

Exemple concret où Semaphore est pertinent

Imaginez une API qui accepte **2 connexions simultanées**.

Chaque thread fait un appel réseau indépendant.

Ils ne modifient pas une même variable Python critique.

Dans ce cas :

- on ne veut pas 1 seul thread

- on veut juste éviter 20 connexions simultanées
 - donc Semaphore(2) est logique
-

8 □ RLock —verrou réentrant

Un Lock ordinaire bloque si le **même thread** essaie de l'acquérir deux fois. Le **RLock** (Reentrant Lock) résout ce problème —le même thread peut l'acquérir plusieurs fois sans se bloquer lui-même.

```
import threading

lock_simple = threading.Lock()
rlock       = threading.RLock()

# Avec Lock ordinaire → DEADLOCK si même thread acquiert 2 fois
def avec_lock():
    with lock_simple:
        print(" Niveau 1 acquis")
        with lock_simple:  # ← bloque indéfiniment ! Le thread s'attend lui-même
            print(" Niveau 2 acquis")  # jamais atteint

# Avec RLock → fonctionne, Le même thread peut entrer plusieurs fois
def avec_rlock():
    with rlock:
        print(" Niveau 1 acquis")
        with rlock:  # ← Le même thread peut rentrer sans blocage
            print(" Niveau 2 acquis")  # □ atteint normalement
    print(" Tout libéré")

# Test RLock uniquement (l'autre bloquerait le programme)
avec_rlock()

Niveau 1 acquis
Niveau 2 acquis
Tout libéré
```

- **RLock maintient un compteur** : même thread → compteur monte (2, 3 ...) sans bloquer ; libéré quand compteur revient à 0
 - **quand utiliser RLock** : une méthode protégée appelle une autre méthode protégée du même objet —cas fréquent dans les classes thread-safe
 - **règle simple** : si vous n'êtes pas sûr, utilisez RLock —il est légèrement plus lent mais évite les deadlocks accidentels
-

9 □ La notion Condition pour faire communiquer des threads

Le type **Condition** permet à un thread d'attendre qu'un autre lui envoie un signal. C'est le pattern **producteur/consommateur** : l'un produit des données, l'autre attend qu'elles soient disponibles.

```
import threading
import time

condition = threading.Condition()
message = None # donnée partagée

def producteur():
    global message
    time.sleep(1) # prépare quelque chose...
    with condition:
        message = "commande_prête"
        print(f" Producteur : message envoyé → '{message}'")
        condition.notify() # réveille le consommateur

def consommateur():
    with condition:
        print(" Consommateur : j'attends un message...")
        condition.wait() # se met en pause jusqu'au notify()
        print(f" Consommateur : message reçu → '{message}' □")

t1 = threading.Thread(target=consommateur)
t2 = threading.Thread(target=producteur)

t1.start()
t2.start()
t1.join()
t2.join()
```

```
Consommateur : j'attends un message...
Producteur : message envoyé → 'commande_prête'
Consommateur : message reçu → 'commande_prête' □
```

- **condition.wait()** : le thread se met en pause **et** libère le lock —il reprend quand **notify()** est appelé
- **condition.notify()** : réveille UN thread en attente —**notify_all()** les réveille tous
- **dans la pratique** : Condition est bas niveau —pour la plupart des cas réels, Queue (section suivante) est plus simple

❑ Queue —la solution recommandée

`queue.Queue` est une **file d'attente thread-safe** —elle intègre automatiquement les mécanismes de synchronisation. C'est l'outil recommandé pour le pattern producteur/consommateur.

```
import threading
import time
from queue import Queue

file = Queue()

def producteur():
    for commande in ["cmd_1", "cmd_2", "cmd_3"]:
        time.sleep(0.3)
        file.put(commande)           # ajoute à la file — thread-safe
        print(f" ❶ Mis en file : {commande}")

def consommateur():
    for _ in range(3):
        commande = file.get()        # attend qu'un élément soit disponible
        time.sleep(0.2)
        print(f" ❷ Traité : {commande}")
        file.task_done()             # signale que cet élément est traité

t1 = threading.Thread(target=producteur)
t2 = threading.Thread(target=consommateur)

t1.start()
t2.start()

file.join()    # attend que tous les éléments aient été traités
t1.join()
t2.join()
print("Pipeline terminé ❸")

❶ Mis en file : cmd_1
❷ Traité : cmd_1
❶ Mis en file : cmd_2
❷ Traité : cmd_2
❶ Mis en file : cmd_3
❷ Traité : cmd_3
Pipeline terminé ❸
```

- **`file.put(x)`** : ajoute un élément —thread-safe, pas besoin de Lock manuel
- **`file.get()`** : récupère un élément —bloque si la file est vide jusqu'à ce

qu'un élément arrive

- **file.task_done()** : indique que le traitement de cet élément est terminé —obligatoire pour que **file.join()** fonctionne
 - **file.join()** : attend que **task_done()** ait été appelé pour chaque élément —garantie que tout a été traité
-

1 ¶1 Introduction aux Coroutines (asyncio)

asyncio offre une **alternative aux threads** pour les opérations I/O. Un seul thread gère de nombreuses tâches en les alternant grâce aux mots-clés **async** et **await**.

L'analogie : un serveur de restaurant. Avec les threads, on embauche un serveur par table. Avec asyncio, un seul serveur gère toutes les tables —il prend une commande, va en cuisine la déposer, revient prendre la commande d'une autre table, et ainsi de suite.

```
import asyncio
```

```
# "async def" : définit une coroutine (pas une fonction ordinaire)
```

```
async def preparer_commande(numero):
```

```
    print(f"  Commande #{numero} : en cours...")
```

```
    await asyncio.sleep(1) # "await" = je lâche le contrôle pendant que j'attends
```

```
    print(f"  Commande #{numero} : ☑ prête")
```

```
async def main():
```

```
    # gather = lancer plusieurs coroutines EN MÊME TEMPS
```

```
    await asyncio.gather(
```

```
        preparer_commande(1),
```

```
        preparer_commande(2),
```

```
        preparer_commande(3),
```

```
    )
```

```
asyncio.run(main())
```

```
Commande #1 : en cours...
```

```
Commande #2 : en cours...
```

```
Commande #3 : en cours...
```

```
Commande #1 : ☑ prête
```

```
Commande #2 : ☑ prête
```

```
Commande #3 : ☑ prête
```

- **async def** : transforme une fonction en **coroutine** —elle peut être mise en pause

- **await** : suspend la coroutine et **rend la main** à la boucle —d'autres coroutines peuvent s'exécuter pendant ce temps
- **asyncio.gather(...)** : démarre plusieurs coroutines **en même temps** —attend que toutes soient finies
- **asyncio.run(main())** : démarre la boucle événementielle —c'est toujours le point d'entrée

1 □2 Coroutine simple —anatomie pas à pas

```
import asyncio

# Une coroutine ressemble à une fonction normale, mais avec async/await
async def dire_bonjour(nom, delai):
    print(f"Bonjour {nom} !")
    await asyncio.sleep(delai) # pause sans bloquer les autres
    print(f"Au revoir {nom} !")

# Preuve : appeler une coroutine sans await ne l'exécute PAS
coro = dire_bonjour("Bechir", 1)
print(f"Type obtenu : {type(coro)}") # <class 'coroutine'> — pas encore exécutée
coro.close() # ferme sans exécuter

# Pour exécuter, il faut await ou asyncio.run
print("\n--- Exécution séquentielle ---")
async def sequentiel():
    await dire_bonjour("Alice", 0.5) # attend la fin avant de continuer
    await dire_bonjour("Bob", 0.5)

asyncio.run(sequentiel()) # 1.0s

print("\n--- Exécution concurrente ---")
async def concurrent():
    await asyncio.gather(
        dire_bonjour("Alice", 0.5), # Les deux démarrent en même temps
        dire_bonjour("Bob", 0.5),
    )

asyncio.run(concurrent()) # 0.5s — deux fois plus rapide !

Type obtenu : <class 'coroutine'>

--- Exécution séquentielle ---
Bonjour Alice !
Au revoir Alice !
```

```
Bonjour Bob !  
Au revoir Bob !
```

```
--- Exécution concurrente ---  
Bonjour Alice !  
Bonjour Bob !  
Au revoir Alice !  
Au revoir Bob !
```

- **sans await** : appeler `dire_bonjour(...)` retourne un objet coroutine —rien ne s'exécute, Python vous avertit avec un `RuntimeWarning`
 - **await dans séquentiel** : attend la fin de chaque coroutine avant de passer à la suivante —1.0s au total
 - **gather dans concurrent** : les deux coroutines démarrent ensemble et partagent le temps d'attente —0.5s au total
-

1 3 Thread vs Coroutine —comparaison directe

Même problème, deux solutions. Observez les différences.

```
import threading  
import asyncio  
import time
```

```
# — Avec Threading  
def tache_thread(nom):  
    print(f" Thread '{nom}' démarre")  
    time.sleep(1)  
    print(f" Thread '{nom}' termine")  
  
debut = time.perf_counter()  
t1 = threading.Thread(target=tache_thread, args=("A",))  
t2 = threading.Thread(target=tache_thread, args=("B",))  
t1.start(); t2.start()  
t1.join(); t2.join()  
print(f"Threading : {time.perf_counter()-debut:.1f}s\n")
```

```
# — Avec Asyncio  
async def tache_async(nom):  
    print(f" Coroutine '{nom}' démarre")  
    await asyncio.sleep(1)  
    print(f" Coroutine '{nom}' termine")  
  
async def main():
```

```

debut = time.perf_counter()
await asyncio.gather(tache_async("A"), tache_async("B"))
print(f"Asyncio : {time.perf_counter()-debut:.1f}s")

asyncio.run(main())

Thread 'A' démarre
Thread 'B' démarre
Thread 'A' termine
Thread 'B' termine
Threading : 1.0s

Coroutine 'A' démarre
Coroutine 'B' démarre
Coroutine 'A' termine
Coroutine 'B' termine
Asyncio : 1.0s

```

Les deux donnent le même résultat. La différence est **sous le capot** :

	Threading	Asyncio
Nb de threads	1 par tâche	1 seul pour tout
Changement de contexte	L'OS décide (imprévisible)	Le code décide (await)
Race conditions	☐ Possible	☐ Rare
Mémoire par tâche	~1 Mo par thread	~1 Ko par coroutine
Limite pratique	~200 threads	10 000+ coroutines
Code bloquant existant	☐ Fonctionne	☐ Doit être réécrit

1 ☐4 Cas réel —appels API en masse

```

import asyncio
import time

# Simule un appel à une API externe (lent par nature)
async def appel_api(identifiant):
    await asyncio.sleep(0.5) # temps de réponse de l'API
    return f"résultat_{identifiant}"

async def main():
    print("=== Séquentiel : un appel après l'autre ===")
    t0 = time.perf_counter()
    resultats = []

```

```

for i in range(5):
    r = await appel_api(i)
    resultats.append(r)
print(f" {resultats}")
print(f" Durée : {time.perf_counter()-t0:.1f}s\n")    # ≈ 2.5s

print("=== Concurrent : tous les appels en même temps ===")
t0 = time.perf_counter()
resultats = await asyncio.gather(*[appel_api(i) for i in range(5)])
print(f" {resultats}")
print(f" Durée : {time.perf_counter()-t0:.1f}s")      # ≈ 0.5s

asyncio.run(main())

=== Séquentiel : un appel après l'autre ===
['résultat_0', 'résultat_1', 'résultat_2', 'résultat_3', 'résultat_4']
Durée : 2.5s

=== Concurrent : tous les appels en même temps ===
['résultat_0', 'résultat_1', 'résultat_2', 'résultat_3', 'résultat_4']
Durée : 0.5s

```

- `*[appel_api(i) for i in range(5)]` : crée une liste de 5 coroutines et les "déplie" comme arguments de `gather`
- **gather retourne les résultats dans l'ordre des arguments** —même si les coroutines se terminent dans un ordre différent
- **gain ×5 ici** : 5 appels de 0.5s → 0.5s au lieu de 2.5s —le gain augmente avec le nombre d'appels

1 □5 CPU-bound —utiliser multiprocessing

Quand la tâche est un **calcul intensif** (pas d'I/O), ni les threads ni `asyncio` n'aident. La solution : `multiprocessing`.

```

import time
import threading
import asyncio
from multiprocessing import Pool

```

```

# -----
# Tâche CPU-bound :
# ici on effectue un calcul volontairement coûteux
# -----
def calcul_lourd(n):

```



```

total = 0

# Boucle de calcul pur
# Il n'y a ni fichier, ni réseau, ni attente I/O
for i in range(n):
    total += i * i

return total

# -----
# Version thread :
# chaque thread lance le même calcul CPU
# les résultats sont stockés dans une liste partagée
# -----
def travail_thread(n, resultats, index):
    resultats[index] = calcul_lourd(n)

# -----
# Version coroutine :
# attention, ici la coroutine n'apporte rien pour du CPU pur
# on l'écrit seulement pour comparer les temps
# -----
async def travail_async(n):
    return calcul_lourd(n)

# -----
# Programme principal
# -----
if __name__ == "__main__":

    # Même charge répétée 4 fois
    # Vous pouvez augmenter la valeur si la machine est très rapide
    valeurs = [8_000_000, 8_000_000, 8_000_000, 8_000_000]

    print("=== Comparaison sur tâche CPU-bound ===\n")

    # =====
    # 1. Exécution séquentielle
    # =====
    debut = time.perf_counter()

    resultats_seq = []
    for n in valeurs:

```

```

        resultats_seq.append(calcul_lourd(n))

fin = time.perf_counter()

print(f"Séquentiel          : {fin - debut:.2f} s")

# =====
# 2. Exécution avec threads
# =====
debut = time.perf_counter()

resultats_threads = [None] * len(valeurs)
threads = []

for index, n in enumerate(valeurs):
    t = threading.Thread(
        target=travail_thread,
        args=(n, resultats_threads, index)
    )
    threads.append(t)
    t.start()

for t in threads:
    t.join()

fin = time.perf_counter()

print(f"Avec threads        : {fin - debut:.2f} s")

# =====
# 3. Exécution avec asyncio
# =====
async def executer_async():
    debut_async = time.perf_counter()

    # On crée 4 coroutines
    # Mais chacune exécute un calcul CPU bloquant
    # Donc il n'y a pas de vrai gain
    taches = [travail_async(n) for n in valeurs]
    resultats_async = await asyncio.gather(*taches)

    fin_async = time.perf_counter()

    print(f"Avec asyncio        : {fin_async - debut_async:.2f} s")

    return resultats_async

```

```

resultats_async = asyncio.run(executer_async())

# =====
# 4. Exécution avec multiprocessing
# =====
debut = time.perf_counter()

# Chaque processus a son propre interpréteur Python
# Donc on obtient du vrai parallélisme CPU
with Pool(processes=4) as pool:
    resultats_process = pool.map(calcul_lourd, valeurs)

fin = time.perf_counter()

print(f"Avec processus      : {fin - debut:.2f} s")

# =====
# Vérification simple
# Toutes les approches doivent produire les mêmes résultats
# =====
meme_resultat = (
    resultats_seq == resultats_threads == resultats_async == resultats_process
)

print(f"\nRésultats identiques : {meme_resultat}")

=== Comparaison sur tâche CPU-bound ===

```

```

Séquentiel      : 2.82 s
Avec threads    : 2.44 s
Avec asyncio    : 2.55 s
Avec processus  : 1.23 s

```

pool.map(f, liste) : distribue les éléments de la liste entre les processus — retourne les résultats dans le même ordre

- **if __name__ == "__main__"** : obligatoire sur Windows —évite que chaque processus enfant relance le script entier au démarrage

1 □6 Bonnes pratiques —résumé opérationnel

```

import threading
import asyncio
from queue import Queue

```

```

# 📌 Pratique 1 : utiliser Queue plutôt que Lock + liste manuelle
# 📌 fragile
donnees = []
lock = threading.Lock()
with lock:
    donnees.append("x")    # gérer le lock partout = risque d'oubli

# 📌 simple et sûr
file = Queue()
file.put("x")    # thread-safe sans lock explicite

# 📌 Pratique 2 : join() toujours présent pour attendre les threads
threads = [threading.Thread(target=lambda: None) for _ in range(3)]
for t in threads: t.start()
for t in threads: t.join()    # sans join(), le main peut finir avant les threads

# 📌 Pratique 3 : asyncio.Semaphore pour limiter les appels API
async def avec_limite():
    sem = asyncio.Semaphore(3)    # max 3 appels simultanés

    async def appel(n):
        async with sem:
            await asyncio.sleep(0.1)
            return n

    return await asyncio.gather(*[appel(i) for i in range(10)])

asyncio.run(avec_limite())

# 📌 Pratique 4 : daemon=True pour les tâches de monitoring
import time
def heartbeat():
    while True:
        time.sleep(5)
        print("📌 service actif")

t = threading.Thread(target=heartbeat, daemon=True)
t.start()
# Pas besoin de t.join() — s'arrête automatiquement avec le programme

```

Guide de décision final :

Ma tâche attend une réponse (réseau, DB, fichier) ?

- Oui, et j'ai peu de tâches (< 100)
 - Threading
- Oui, et j'ai beaucoup de tâches (100+)
 - asyncio
- Non, c'est du calcul pur (ML, chiffrement, traitement)
 - multiprocessing

J'ai des données partagées entre threads ?

- Accès exclusif requis (une seule opération à la fois)
 - Lock
- Limiter le nombre d'accès simultanés
 - Semaphore
- Passer des données d'un thread à l'autre
 - Queue (recommandé par défaut)

□ Récapitulatif — les 5 concepts clés

Thread : start() + join() · **Race condition** : données partagées sans protection → résultat imprévisible · **Lock** : with lock : protège une section critique · **GIL** : threads OK pour I/O, pas pour CPU · **asyncio** : async/await + gather pour l'I/O concurrent dans un seul thread

Partie II les 20 questions intelligentes

1) Pourquoi un thread peut accélérer une tâche I/O mais pas une tâche CPU ?

Énoncé

Expliquer la différence entre une tâche I/O et une tâche CPU, et montrer pourquoi les threads aident surtout pour I/O.

Solution

"""

Explication

- Une tâche I/O attend (réseau, disque, base) : pendant l'attente, le CPU peut exécuter d'autres tâches.
 - Une tâche CPU calcule en continu : en CPython, le GIL limite l'exécution simultanée de threads.
 - Démo : I/O simulée avec sleep (threads efficaces), CPU simulé par calcul (threads peu efficaces).
 """

```

import time
from threading import Thread

def io_task(n: int):
    # Simule une attente I/O
    time.sleep(1)
    print(f"I/O {n} terminé")

def cpu_task(n: int):
    # Simule un calcul CPU
    total = 0
    for i in range(3_000_000):
        total += i
    print(f"CPU {n} terminé, total={total}")

def demo_io():
    debut = time.perf_counter()
    threads = [Thread(target=io_task, args=(i,)) for i in range(3)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    fin = time.perf_counter()
    print("Temps I/O threads:", round(fin - debut, 2), "s")

def demo_cpu():
    debut = time.perf_counter()
    threads = [Thread(target=cpu_task, args=(i,)) for i in range(3)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    fin = time.perf_counter()
    print("Temps CPU threads:", round(fin - debut, 2), "s")

demo_io()
demo_cpu()

```

2) Pourquoi join est obligatoire pour mesurer un temps correct ?

Énoncé

Que se passe-t-il si on mesure le temps sans attendre la fin des threads ?

Solution

```
"""
Explication
- start lance le thread, mais le main continue immédiatement.
- Sans join, la mesure s'arrête avant la fin réelle des traitements.
- Démo : comparaison mesure avec et sans join.
"""

import time
from threading import Thread

def travail():
    time.sleep(1)
    print("Travail terminé")

# Sans join
debut = time.perf_counter()
t = Thread(target=travail)
t.start()
fin = time.perf_counter()
print("Mesure sans join:", round(fin - debut, 4), "s")

# Avec join
debut = time.perf_counter()
t = Thread(target=travail)
t.start()
t.join()
fin = time.perf_counter()
print("Mesure avec join:", round(fin - debut, 4), "s")
```

3) Race condition : pourquoi un résultat peut changer à chaque exécution ?

Énoncé

Montrer une incrémentation concurrente qui donne un résultat instable.

Solution

```
"""
Explication
- Une opération compteur += 1 n'est pas atomique : lecture, calcul, écriture.
- Deux threads peuvent lire la même valeur et écraser l'incrément de l'autre.
- Démo : Lancer plusieurs fois pour observer la variabilité.
"""

from threading import Thread

def incrementer(compteur: list[int], n: int):
    for _ in range(n):
        compteur[0] += 1

for essai in range(3):
    compteur = [0]
    t1 = Thread(target=incrementer, args=(compteur, 200_000))
    t2 = Thread(target=incrementer, args=(compteur, 200_000))
    t1.start()
    t2.start()
    t1.join()
    t2.join()
    print("Essai", essai + 1, "résultat:", compteur[0], "attendu:", 400_000)
```

4) Lock : comment garantir la cohérence d'une section critique ?

Énoncé

Corriger la question précédente avec un verrou.

Solution

```
"""
Explication
- Un verrou protège la section critique : un seul thread à la fois.
- with lock est préférable : acquisition et libération garanties.
"""

from threading import Thread, Lock

def incremater(compteur: list[int], n: int, lock: Lock):
    for _ in range(n):
        with lock:
            compteur[0] += 1

compteur = [0]
lock = Lock()

t1 = Thread(target=incremater, args=(compteur, 200_000, lock))
t2 = Thread(target=incremater, args=(compteur, 200_000, lock))

t1.start()
t2.start()
t1.join()
t2.join()

print("Résultat:", compteur[0], "attendu:", 400_000)
```

5) Deadlock : comment le provoquer facilement ?

Énoncé

Créer un deadlock avec deux verrous acquis dans des ordres opposés.

Solution

```
"""
Explication
- Un deadlock survient quand :
    1) T1 détient lockA et attend lockB
    2) T2 détient lockB et attend lockA
- Démo : deux threads bloqués.
- Astuce : exécuter puis interrompre manuellement si blocage.
"""
```

```

import time
from threading import Thread, Lock

lock_a = Lock()
lock_b = Lock()

def tache_1():
    with lock_a:
        print("T1 a lockA")
        time.sleep(0.2)
        print("T1 attend lockB")
        with lock_b:
            print("T1 a lockB")

def tache_2():
    with lock_b:
        print("T2 a lockB")
        time.sleep(0.2)
        print("T2 attend lockA")
        with lock_a:
            print("T2 a lockA")

t1 = Thread(target=tache_1)
t2 = Thread(target=tache_2)

t1.start()
t2.start()
t1.join(timeout=1)
t2.join(timeout=1)

print("Si les threads ne terminent pas, deadlock probable.")

```

6) Comment éviter un deadlock proprement ?

Énoncé

Montrer une stratégie simple et robuste.

Solution

"""
Explication

*- Stratégie : imposer un ordre global d'acquisition (toujours lockA puis lockB).
- Même si plusieurs threads, l'ordre unique supprime le cycle d'attente.*

"""

```
import time
from threading import Thread, Lock

lock_a = Lock()
lock_b = Lock()

def worker(n: int):
    with lock_a:
        time.sleep(0.1)
    with lock_b:
        print("Worker", n, "a acquis lockA puis lockB")

threads = [Thread(target=worker, args=(i,)) for i in range(3)]
for t in threads:
    t.start()
for t in threads:
    t.join()

print("Terminé sans deadlock.")
```

7) Semaphore : pourquoi ce n'est pas un Lock ?

Énoncé

Montrer la différence entre accès exclusif et limitation de concurrence.

Solution

"""

Explication

- Lock : 1 seul thread à la fois (exclusif).*
- Semaphore(k) : jusqu'à k threads simultanément.*
- Démo : 2 threads peuvent entrer en même temps, le 3e attend.*

"""

```
import time
from threading import Thread, Semaphore

sem = Semaphore(2)
```

```

def requete(n: int):
    print("Requête", n, "attend")
    with sem:
        print("Requête", n, "entre")
        time.sleep(1)
        print("Requête", n, "sort")

threads = [Thread(target=requete, args=(i,)) for i in range(1, 5)]
for t in threads:
    t.start()
for t in threads:
    t.join()

```

8) RLock : à quoi ça sert concrètement ?

Énoncé

Montrer un cas où un Lock simple casse le programme, mais RLock fonctionne.

Solution

```

"""
Explication
- Un Lock simple ne peut pas être repris par le même thread : blocage.
- Un RLock permet au même thread de ré-acquérir le verrou plusieurs fois.
- Démo : fonction qui appelle une autre fonction verrouillée.
"""

```

```

from threading import RLock

lock = RLock()
solde = [100]

def crediter(montant: int):
    with lock:
        solde[0] += montant

def transferer(montant: int):
    with lock:
        # Appel interne qui reprend le même verrou
        crediter(montant)
        print("Solde après transfert:", solde[0])

```

```
transférer(50)
```

9) Condition : pourquoi wait doit être dans une boucle ?

Énoncé

Montrer qu'un réveil peut arriver sans que la condition logique soit vraie.

Solution

```
"""
Explication
- Une Condition coordonne des threads, mais un réveil ne garantit pas l'état attendu.
- Bonne pratique : attendre dans une boucle while sur une condition métier.
- Démo : producteur/consommateur sécurisé.
"""

import time
from threading import Thread, Condition

condition = Condition()
buffer: list[int] = []

def producteur():
    for x in [10, 20, 30]:
        time.sleep(0.2)
        with condition:
            buffer.append(x)
            print("Produit:", x)
            condition.notify()

def consommateur():
    for _ in range(3):
        with condition:
            while not buffer:
                condition.wait()
            x = buffer.pop(0)
            print("Consommé:", x)

t1 = Thread(target=producteur)
t2 = Thread(target=consommateur)
```

```
t1.start()
t2.start()
t1.join()
t2.join()
```

10) Queue : pourquoi c'est souvent meilleur que Lock + liste ?

Énoncé

Montrer un producteur/consommateur simple, sans verrou explicite.

Solution

```
"""
```

Explication

- Queue est thread-safe : pas besoin de Lock manuel.
- get bloque jusqu'à disponibilité, task_done signale la fin du traitement.
- Démo : pipeline simple.

```
"""
```

```
import time
from threading import Thread
from queue import Queue

q: Queue[int] = Queue()

def producteur():
    for x in [1, 2, 3]:
        time.sleep(0.2)
        q.put(x)
        print("Produit:", x)

def consommateur():
    for _ in range(3):
        x = q.get()
        print("Consommé:", x)
        q.task_done()

t1 = Thread(target=producteur)
t2 = Thread(target=consommateur)
t1.start()
t2.start()
```

```
q.join()
t1.join()
t2.join()

print("Pipeline terminé.")
```

11) Predict : quel est l'ordre d'affichage des threads ?

Énoncé

Prédire si l'ordre des prints est déterministe.

Solution

```
"""
Explication
- L'ordre dépend du scheduler et du timing : non déterministe.
- Démo : lancer plusieurs fois et observer que l'ordre change.
"""

import time
from threading import Thread

def tache(n: int):
    time.sleep(0.01 * (3 - n))
    print("Thread", n)

threads = [Thread(target=tache, args=(i,)) for i in [1, 2, 3]]
for t in threads:
    t.start()
for t in threads:
    t.join()

print("Relancer pour observer l'ordre.")
```

12) Daemon thread : quel risque réel en production ?

Énoncé

Montrer qu'un daemon peut être interrompu sans terminer.

Solution

```
"""
Explication
- Un thread daemon est stoppé quand le main se termine.
- Risque : Logs incomplets, flush non fait, écritures perdues.
"""

import time
from threading import Thread

def journaliser():
    for i in range(10):
        time.sleep(0.2)
        print("Log", i)

t = Thread(target=journaliser, daemon=True)
t.start()

time.sleep(0.6)
print("Main termine, daemon peut être coupé avant la fin.")
```

13) Thread subclass : pourquoi préférer target dans Thread ?

Énoncé

Comparer deux styles et montrer le style le plus simple à maintenir.

Solution

```
"""
Explication
- Hériter de Thread est utile si on encapsule état + comportement.
- Utiliser target est plus simple pour des tâches courtes.
```


- D  mo : m  me job avec les deux approches.
"""

```
import time
from threading import Thread

def job(n: int):
    time.sleep(0.1)
    print("Job target:", n)

class Worker(Thread):
    def __init__(self, n: int):
        super().__init__()
        self.n = n

    def run(self):
        time.sleep(0.1)
        print("Job subclass:", self.n)

t1 = Thread(target=job, args=(1,))
t2 = Worker(2)

t1.start()
t2.start()
t1.join()
t2.join()
```

14) asyncio : pourquoi await est obligatoire ?

  nonc  

Montrer que cr  er une coroutine sans await ne l'ex  cute pas.

Solution

```
"""
Explication
- Une fonction async retourne une coroutine.
- Sans await (ou asyncio.run / gather), elle ne s'ex  cute pas.
"""
```

```
import asyncio
```

```

async def f():
    print("Début f")
    await asyncio.sleep(0.1)
    print("Fin f")

# Création sans exécution
coro = f()
print("Objet coroutine créé:", coro)

# Exécution réelle
asyncio.run(f())

```

15) Predict : que renvoie gather si une coroutine échoue ?

Énoncé

Montrer le comportement par défaut et la variante qui récupère les exceptions.

Solution

```

"""
Explication
- Par défaut, gather propage la première exception.
- Avec return_exceptions=True, on récupère résultats + exceptions.
"""

import asyncio

async def ok():
    await asyncio.sleep(0.1)
    return "OK"

async def ko():
    await asyncio.sleep(0.05)
    raise ValueError("Erreur métier")

async def main():
    try:
        res = await asyncio.gather(ok(), ko())
        print("Résultats:", res)
    except Exception as e:
        print("Exception propagée:", type(e).__name__, str(e))

```

```
res = await asyncio.gather(ok(), ko(), return_exceptions=True)
print("Mode return_exceptions:", res)

asyncio.run(main())
```

16) Bloquer la boucle asyncio : comment détecter le bug ?

Énoncé

Montrer un code async qui semble correct mais bloque tout.

Solution

```
"""
Explication
- time.sleep bloque le thread et donc la boucle event loop.
- Il faut utiliser asyncio.sleep.
- Démo : une tâche périodique est gelée si sleep bloquant existe.
"""

import asyncio
import time

async def tick():
    for i in range(5):
        await asyncio.sleep(0.1)
        print("Tick", i)

async def mauvais():
    time.sleep(0.5)
    print("Mauvais terminé")

async def bon():
    await asyncio.sleep(0.5)
    print("Bon terminé")

async def main():
    print("Cas bloquant")
    await asyncio.gather(tick(), mauvais())

    print("Cas non bloquant")
```

```
        await asyncio.gather(tick(), bon())

    asyncio.run(main())
```

17) asyncio Semaphore : comment limiter la concurrence proprement ?

Énoncé

Simuler 10 appels I/O mais en limiter 3 simultanément.

Solution

```
"""
Explication
- Limiter la concurrence évite de saturer API, DB, réseau.
- asyncio.Semaphore contrôle le nombre de tâches simultanées.
"""

import asyncio

sem = asyncio.Semaphore(3)

async def appel(n: int):
    async with sem:
        print("Start", n)
        await asyncio.sleep(0.2)
        print("End", n)
        return n

async def main():
    tasks = [appel(i) for i in range(1, 11)]
    res = await asyncio.gather(*tasks)
    print("Total résultats:", len(res))

asyncio.run(main())
```

18) Thread + asyncio : comment exécuter une fonction bloquante sans geler la boucle ?

Énoncé

Montrer l'utilisation d'un exécuteur pour isoler du code bloquant.

Solution

```
"""
Explication
- Certaines bibliothèques sont bloquantes (I/O sync).
- run_in_executor exécute la fonction dans un thread, sans bloquer la boucle.
"""

import asyncio
import time

def bloquant():
    time.sleep(0.4)
    return "Résultat bloquant"

async def main():
    loop = asyncio.get_running_loop()
    print("Avant exécuteur")
    res = await loop.run_in_executor(None, bloquant)
    print("Après exécuteur:", res)

asyncio.run(main())
```

19) Timeout : comment protéger un appel async qui peut se bloquer ?

Énoncé

Montrer un timeout clair et observable.

Solution

```
"""
Explication
- wait_for impose une limite : si dépassée, TimeoutError.
"""
```

- Utile pour appels réseau instables.
"""

```
import asyncio

async def lent():
    await asyncio.sleep(1)
    return "OK"

async def main():
    try:
        res = await asyncio.wait_for(lent(), timeout=0.2)
        print(res)
    except asyncio.TimeoutError:
        print("Timeout déclenché")

asyncio.run(main())
```

20) Mini défi : pipeline e-commerce (ingestion, traitement, sortie)

Énoncé

Construire un mini pipeline : un producteur crée des commandes, un consommateur traite, puis confirme.

Solution

"""
Explication
- Modèle très courant : ingestion → file → workers → ack.
- Ici : Queue thread-safe + deux workers.
- Démo : affichage du flux complet.
"""

```
import time
from threading import Thread
from queue import Queue

q: Queue[dict] = Queue()

def producteur():
    commandes = [
```

```

        {"id": 101, "montant": 50},
        {"id": 102, "montant": 120},
        {"id": 103, "montant": 30},
        {"id": 104, "montant": 90},
    ]
    for c in commandes:
        time.sleep(0.1)
        q.put(c)
        print("Reçu commande:", c["id"])
    print("Producteur terminé")

def worker(n: int):
    while True:
        commande = q.get()
        if commande is None:
            q.task_done()
            print("Worker", n, "stop")
            return

        # Traitement métier simulé
        time.sleep(0.2)
        decision = "ACCEPTÉE" if commande["montant"] <= 100 else "REVUE"
        print("Worker", n, "traite", commande["id"], "=>", decision)

        q.task_done()

t_prod = Thread(target=producteur)
t_w1 = Thread(target=worker, args=(1,))
t_w2 = Thread(target=worker, args=(2,))

t_prod.start()
t_w1.start()
t_w2.start()

t_prod.join()
q.join()

# Stop des workers via sentinelles
q.put(None)
q.put(None)
q.join()

t_w1.join()
t_w2.join()

print("Pipeline complet.")

```